

GOETHE-UNIVERSITÄT FRANKFURT
AM MAIN

FACHBEREICH INFORMATIK

Instruction Robustness in
RAG/GraphRAG Agent
Pipelines

Bachelor Thesis

Name: Christian Block

Matriculation Number: 6522508

First examiner: Prof. Dr. Visvanathan Ramesh

Date of Submission:

Erklärung zur Abschlussarbeit

gemäß § 35, Abs. 16 der Ordnung für den Bachelorstudiengang Informatik vom 17. Juni 2019:

Hiermit erkläre ich (Nachname, Vorname) **Block, Christian**, dass ich die vorliegende Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel verfasst habe. Ich bestätige außerdem, dass die vorliegende Arbeit nicht, auch nicht auszugsweise, für eine andere Prüfung oder Studienleistung verwendet wurde. Zudem versichere ich, dass alle eingereichten schriftlichen gebundenen Versionen meiner vorliegenden Bachelorarbeit mit der digital eingereichten elektronischen Version meiner Bachelorarbeit übereinstimmen.

Frankfurt am Main, den DATUM _____

Unterschrift der/des Studierenden

Abstract

Retrieval-augmented generation (RAG) combines information retrieval with language generation, such that a language model can condition its output on external and updatable evidence. The system prompt specifies the instructions, constraints, and output format the generator should adhere. Because it is written in natural language, its effect on system behaviour is hard to predict and hard to verify. Prior work on prompt sensitivity in language models has studied that effect outside retrieval-augmented systems. This thesis addresses that gap by asking how far systematic changes in the system prompt influence the behaviour and outcomes of a RAG system when the user input, knowledge base, retrieval mechanism, and underlying language model are held fixed.

The thesis builds a formal model of a RAG system on the formulation of Lewis et al. (2020). A retriever assigns probabilities $p_\eta(z \mid q, D)$ to documents from the knowledge base; a generator produces the answer token by token. The system prompt S enters as an explicit variable, $p_\theta(y_i \mid q, z, y_{1:i-1}, S)$, and is modelled as a set of checkable requirements Σ written in a wording φ , such that $S = \rho(\Sigma, \varphi)$. Each of three baseline prompts is written in four wordings: its initial form and three rewordings, which change the vocabulary, the sentence structure or the layout. Every rewording leaves the requirements fixed and changes only φ .

Changing the system prompt is an intervention on the system. The properties a good answer must have are fixed beforehand: grounded in

the retrieved evidence, relevant to the question asked, correct against the reference answer, and obedient to the instructions it was given. The knowledge base and the query set come from an established question-answer dataset, so that the measurement is reproducible and its results comparable with published work. Only the system prompt is manipulated. The language model, the embedding model, the vector index, the chunking strategy and the retrieval depth stay fixed, to ensure that the passages that reach the generator are identical in every condition and any difference in the output originates at the generator alone. Generations are repeated to account for decoding randomness.

Rewording the prompts changed what the system produced. The null hypothesis of equal behaviour across wordings is rejected. The effects are largest on instruction adherence, where a rewording moves compliance by up to 46 points in either direction, and they hold at both evidence levels. Faithfulness and answer relevance did not move.

Acknowledgments

I want to express my sincere gratitude to Professor Ramesh for his guidance and mentorship throughout this thesis and Professor Gao for being the second supervisor of this thesis.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Research Objective	2
1.3	Thesis Structure	2
	Notation	3
2	Preliminaries	6
2.1	Path towards RAG	6
2.2	Components of RAG	7
2.2.1	Knowledge Base	7
2.2.2	Retriever	9
2.2.3	Generator	10
2.2.4	System Prompt	12
2.3	Retrieval-Augmented Generation versus Fine-Tuning .	13
2.4	Use-Cases of RAG	14
2.5	Advanced RAG Architectures	15
3	Literature Review	17
3.1	Prompt Sensitivity and Instruction-Following in Lan- guage Models	17
3.2	Retrieval-Augmented Generation: Architecture, Evalu- ation, and Failure Modes	19
3.3	Synthesis and Research Gap	21
4	Formal Model of a RAG System	22
4.1	Modelling Objective	22

4.2	Probabilistic Retrieval	23
4.3	Prompt-Conditioned Generation	23
4.4	Marginalizing over Retrieved Documents	24
4.4.1	RAG-Sequence	24
4.4.2	RAG-Token	25
4.5	The System Prompt: Requirements and Wording . . .	25
4.6	The System Prompt as an Intervention	28
4.7	The Causal Effect	29
4.8	Grounding the Knowledge Base in Established Dataset	30
4.9	What Counts as a Good Answer	31
5	Experimental Setup	33
5.1	Research Question and Hypothesis	33
5.2	Overview of the Experiment	34
5.3	System-Prompt Conditions	35
5.4	Variables and Controlled Factors	41
5.5	Knowledge Base and Query Set	43
5.6	Experimental Procedure	44
5.6.1	Stage 1: Retrieval and the Retrieval Gate . . .	44
5.6.2	Stage 2: Generation	46
5.6.3	The Pipeline	47
5.7	Evaluation	47
5.7.1	Five Measures for Four Properties	48
5.7.2	The Judge	51
5.7.3	Estimating the Effect of a Rewording	51
6	Results and Discussion	53
6.1	How the System Behaved Before Any Comparison . . .	53
6.2	The Effect of Rewording	55
6.2.1	Instruction Adherence	55

6.2.2	Faithfulness	56
6.2.3	Answer Relevance	56
6.2.4	Correctness	56
6.3	The Verdict on the Hypothesis	58
6.4	Summary of Findings	59

1 Introduction

1.1 Background and Motivation

Retrieval-Augmented Generation (RAG) systems combine information retrieval with language generation. This design addresses a limit of purely parametric large language models (LLMs), whose knowledge is fixed at training time and cannot be updated without retraining or fine-tuning. Lewis et al. (2020) introduced RAG by combining a retriever with a pretrained sequence-to-sequence generator, to enable the model to base its output on external and updatable evidence. A RAG system takes the user query, retrieves relevant information from an external knowledge base and gives that information to a language model with instructions for generating the final response.

The system prompt connects the intended system behaviour to the observable output. The retrieved evidence varies with the query and the knowledge base. The system prompt specifies how the model should behave, which instructions and constraints apply, and which output format it should produce. Because it is written in natural language, its effect on the system behaviour is difficult to predict. Its influence therefore has to be examined through controlled experiments.

1.2 Research Objective

The central research question is the following:

To what extent do systematic changes in the system prompt influence the behaviour and outcomes of a RAG system when the user input, knowledge base, retrieval mechanism, and underlying language model are held fixed?

Answering it requires a formal model of the RAG system that identifies the system prompt as a controlled parameter, and an experimental design that holds every other source of variation fixed.

This thesis builds both for the simplest retrieval-augmented architecture. In that architecture the system prompt reaches the answer along one path. This is what makes its effect identifiable.

1.3 Thesis Structure

This thesis has six chapters. Chapter 2 elaborates how retrieval-augmented approaches to language generation developed and introduces the components of a RAG system. Chapter 3 reviews the literature and identifies the research gap. Chapter 4 builds the formal model used here: it takes the probabilistic formulation of Lewis et al. (2020) and extends it so that the system prompt becomes an explicit variable of the generator, grounds the knowledge base, and defines what counts as a good answer. Chapter 5 describes the experimental setup. Chapter 6 presents and discusses the results, summarises the findings and states what the design does not establish. The Notation section at the end of this chapter lists the symbols used later.

Notation

The notation follows Lewis et al. (2020) for the RAG system and the intervention calculus of Pearl (2009) for the causal statements.

The system and its components

- $q \in \mathcal{Q}$ denotes a user query, and $P_{\mathcal{Q}}$ the distribution over the query space $\mathcal{Q}$ from which queries are drawn.
- $D = \{d_1, d_2, \dots, d_N\}$ denotes the knowledge base, where each d_i is a document and N is the number of documents.
- S denotes a system prompt.
- C denotes the retrieved context supplied to the language model.
- $y = (y_1, \dots, y_L)$ denotes the generated system output, a sequence of L tokens; $y_{1:i-1}$ denotes the tokens produced before position i .
- R and G denote the retrieval and generation functions of the pipeline, and χ the chunking strategy by which documents are divided into retrievable units.
- $v \in \mathcal{V}$ denotes a token of the vocabulary $\mathcal{V}$, u_v the score the generator assigns it, and $T > 0$ the decoding temperature.

The probabilistic model

- $z \in D$ denotes a single retrieved document, and $\mathcal{Z}_K(q, D)$ the K documents with the highest retrieval probability.

- $p_\eta(z \mid q, D)$ denotes the retrieval distribution with parameters η , and $p_\theta(y_i \mid q, z, y_{1:i-1}, S)$ the generator with parameters θ .
- $p_{\text{RAG}}(y \mid q, D, S)$ denotes the distribution over outputs.
- Y denotes the output viewed as a random variable, $M(Y)$ a measured property of it, and ε the randomness introduced by decoding.

The system prompt and its variation

- Σ denotes the set of requirements a system prompt states. Each requirement is a property an answer either has or does not have.
- φ denotes the wording in which those requirements are expressed, and $S = \rho(\Sigma, \varphi)$ denotes the resulting system prompt.
- $S_{b,0}, \dots, S_{b,3}$ denote the four prompt conditions of baseline $b \in \{A, B, C\}$ with $S_{b,0}$ the canonical form of that baseline.

Causal intervention

- $\text{do}(S = S_j)$ denotes the intervention that sets the system prompt to condition j .
- τ_j denotes the average causal effect of condition S_j relative to the baseline S_0 , taken over the query distribution P_Q , and $\hat{\tau}_j$ its estimate from the sample.
- $\mathbb{E}[\cdot]$ denotes expectation, and $\hat{\cdot}$ marks a quantity estimated from the sample.

Experimental design

- c_{ik} denotes the k -th context given to query q_i , with c^{gold} and c^{ret} the two evidence levels.
- $\mathcal{Q}_{\text{direct}}$ and $\mathcal{Q}_{\text{multi-hop}}$ denote the two query categories, and N_Q the size of the query set.
- A_q denotes the reference answers annotated for query q , against which correctness is measured.
- I denotes the set of queries in one cell of the design, and $\bar{M}_{ij}$ the mean of a measure over the n repetitions of query i under prompt condition j .
- The indices are $i \in \{1, \dots, N_Q\}$ over queries, $j \in \{0, \dots, 3\}$ over prompt conditions, $k \in \{1, 2\}$ over evidence levels, and $r \in \{1, \dots, n\}$ over the n repetitions of each cell.

2 Preliminaries

Chapter 1 established that the system prompt is the interface through which a RAG system’s intended behaviour is specified, and asked how systematically varying that interface changes the system behaviour and outcomes. A formal answer requires an account of the components of a RAG system and of the reasons behind its design.

2.1 Path towards RAG

Question-answering systems have always drawn on an external store rather than on what a model holds internally. Early systems relied on hand-crafted rules and pattern matching over the input question, and mapped recognised syntactic patterns onto retrieval or answer-generation procedures (Pandya and Bhatt, 2021). Statistical and later neural methods replaced those rule-based components, shifting the dominant approach towards learning the mapping from question to answer from data.

Large language models generate text from parametric knowledge alone. The information are encoded in their weights during training (Wang et al., 2024). This leads to two problems. First, a model’s knowledge is fixed at the point its training data was collected. Anything that appears after that point is unknown to the model unless it is retrained or fine-tuned. Second, purely parametric models are prone to hallucination: nothing in their design forces an output to trace

back to a verifiable source, so they can generate plausible-sounding statements that no evidence supports.

Both problems motivate giving a language model access to a knowledge base at inference time instead of relying only on what is stored in its parameters (Lin et al., 2025). Lewis et al. (2020) formalised this idea as RAG. RAG combines a retriever, which selects relevant passages from a document collection, with a pretrained generator, which conditions its output on those retrieved passages in addition to the input query.

2.2 Components of RAG

A RAG system has four components: the knowledge base the evidence comes from, the retriever that selects relevant evidence for a given query, the generator that produces a response from the query and the retrieved evidence, and the system prompt that specifies how the generator should behave. Chapter 4 treats them formally. Figure 2.1 shows how they fit together and which information passes between them.

2.2.1 Knowledge Base

The knowledge base $D = \{d_1, d_2, \dots, d_N\}$ is the collection of documents the evidence comes from. In most RAG systems, documents are first split into smaller passages, called chunks. Chunking is needed because a language model can only take in a limited amount of context.

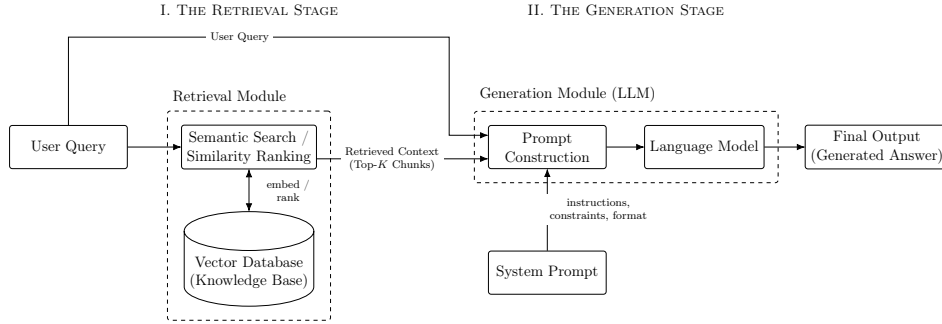


Figure 2.1: Architecture of a RAG system showing the four components introduced. The retrieval stage matches the user query against a vector database and returns the top- K chunks. The generation stage combines those chunks with the user query and the system prompt into a single prompt and produces the answer.

An embedding model maps each chunk to a high-dimensional vector, so that chunks with similar meaning end up close together in the embedding space. Representing text as vectors and measuring relatedness by geometric proximity goes back to the vector space model of Salton et al. (1975). Similarity between a query vector and a chunk vector is usually measured by the cosine of the angle between them.

The embeddings are stored in a vector database, which supports fast nearest-neighbour search over large collections of chunks. Comparing a query against every stored vector is exact but becomes too slow as the collection grows. Therefore, vector databases index the vectors and use approximate nearest-neighbour search. The most widely used index of this kind is the hierarchical navigable small world graph of Malkov and Yashunin (2020).

Pan et al. (2024) survey how such systems differ in indexing, query processing and storage. They also differ in how they are deployed: FAISS is a library embedded directly in an application, with fine-grained control over the index structure (Johnson et al., 2021), whereas Chroma, the `pgvector` extension of PostgreSQL and the managed service Pinecone package the same functionality as a database. This

thesis uses FAISS, for the reason given in Section 5.6.

2.2.2 Retriever

Given a user query q and a knowledge base D , the retriever selects the chunks that are relevant for answering q . Chapter 4 formalises this as a probability distribution $p_\eta(z \mid q, D)$ over relevant documents. Two approaches are common, and they differ in how they match a query to a passage.

2.2.2.1 Dense Retrieval

Dense retrieval embeds the query with the same embedding model used for the knowledge base and ranks chunks by the similarity between the query vector and each chunk vector. Because the embedding is learned, two passages that express the same content in different words end up close together. Hence, a query can match a chunk with which it shares no vocabulary. This is the mechanism the original RAG architecture of Lewis et al. (2020) relies on.

2.2.2.2 Sparse Retrieval

Sparse retrieval scores chunks by term overlap with the query. The standard method is BM25 (Robertson and Zaragoza, 2009). BM25 weights each matching term by how often it occurs in the chunk, how rare it is across the collection, and how long the chunk is. Common words therefore contribute little, and long chunks are not favoured only for containing more words. The representation is sparse since a chunk is described by the vocabulary it contains. BM25 requires no

training and remains a strong baseline.

2.2.2.3 Retrieval Depth

Whichever method is used, the top- K ranked chunks form the retrieved context. Retrieval quality depends on both how many chunks are retrieved and how precise the ranking is. Too few chunks risk missing needed evidence, while too many risk adding irrelevant information to the generation step. The order of the chunks also matters. Liu et al. (2024) report that answer accuracy depends on where the relevant passage sits within the context. This thesis fixes the retriever, the chunking and K across all experimental conditions. Therefore, changes in the output cannot be traced back to retrieval.

2.2.3 Generator

The generator is the large language model that produces the final response. Such a model is a neural network trained on a large text corpus to predict the next token in a sequence. It generates text by autoregressive decoding. Its parameters, written θ in Chapter 4, encode what the model learned during training. This encoded knowledge is often called parametric memory.

Within a RAG pipeline the model reads the retrieved chunks, selects the parts that answer the query, resolves references between them, and writes the result in natural language (Lewis et al., 2020).

The three inputs reach the model in separate turns. The system prompt is supplied in the system role, and the retrieved context and the query in the user role. The model’s chat template then assembles the turns into a single token sequence, with the system prompt first,

so that it shapes how everything after it is read. Chapter 4 formalises the generator as $p_\theta(y_i \mid q, z, y_{1:i-1}, S)$, the probability of the next token given the query, a retrieved document, the tokens already produced and the system prompt. Because the query, the retrieved context and the system prompt together determine the model’s input, changing any one of them can change the output. The others must therefore be held fixed if a change in behaviour is to be attributed to the system prompt alone.

2.2.3.1 Decoding and the Temperature Parameter

Generation is not deterministic by default. At each step the model produces a score u_v for every token v in the vocabulary $\mathcal{V}$, and the softmax function, with a temperature parameter $T > 0$, turns these scores into a probability distribution:

$$p(v) = \frac{\exp(u_v/T)}{\sum_{v' \in \mathcal{V}} \exp(u_{v'}/T)}. \quad (2.1)$$

The temperature rescales the scores before they are normalised, and it controls how much probability mass is left for tokens the model considers less likely (Holtzman et al., 2020). As T approaches zero the distribution concentrates on the highest-scoring token, and decoding becomes effectively deterministic. As T grows the distribution flattens, lower-scoring tokens become reachable, and the output becomes more varied across runs.

For open-ended writing this stochasticity is useful, where as for a RAG system it is mostly harmful. A token that the model scores as unlikely given the retrieved context is one that the evidence does not support well. Consequently, raising the temperature increases the chance of producing a statement the retrieved documents do not

contain. High temperature also makes the system harder to evaluate, since the same query and the same evidence no longer produce the same answer. RAG systems intended to be faithful to their sources are therefore usually run at a low temperature. This thesis holds T fixed across all experimental conditions at the API default of 1.0 in Equation (2.1). That is higher than a system built for faithfulness would choose. Holding it constant is what the comparison between prompts requires. The noise that remains at that setting is the reason each condition is repeated several times in Chapter 5.

2.2.4 System Prompt

The system prompt S specifies the instructions, constraints and output requirements the generator should follow. The system prompt has two functions. The first is to define the role the RAG system occupies, the audience it addresses, and the format its output takes. The second is to impose constraints on that behaviour, of which the grounding constraint is the most important. A grounding constraint restricts the model to the retrieved context and specifies what to do when that context does not suffice. Listing 2.1 illustrates both in a minimal example.

```
You are a customer service assistant for a parcel delivery
company.

GROUNDING RULE
Answer only from the context provided below. Do not use
information
from your training data, even where you are confident that it
is
```

```

correct.

INSUFFICIENT EVIDENCE
If the context does not contain the answer, reply with
    exactly:
The provided documents do not contain this information.

FORMAT
Answer in at most three sentences. Cite the identifier of
    every
document you use, in square brackets.

```

Listing 2.1: A minimal system prompt combining an operational scope with two constraints.

The first line fixes the operational scope. The two blocks that follow are constraints. One restricts which sources a fact may come from, the other defines the response for the case in which no information exists. The final block constrains form. Each of these instructions is stated so that it can be checked against an output. That is the property Chapter 4 requires of the requirement set Σ .

2.3 Retrieval-Augmented Generation versus Fine-Tuning

Fine-tuning is the other established way of adapting a general-purpose language model to a specific application. Fine-tuning continues training on domain-specific data and changes the model parameters θ , so what the model learns enters the parametric memory described in Section 2.2.3. RAG leaves θ untouched and changes what is placed in

the context window at inference time. Fine-tuning requires a training run over the model’s parameters whenever the underlying information changes. Adding a document to a knowledge base requires only that the new chunks be embedded and indexed (Cheerla, 2025). Wang et al. (2024) survey the difficulty of locating and editing individual facts inside model parameters. This matters wherever information is corrected or retracted, or must be deleted on request. It is also one reason retrieval is preferred in regulated settings, where sensitive data can remain in a controlled external store instead of entering the parameters (He et al., 2025). Because a RAG answer is produced from a specific set of retrieved chunks, each claim in the output can in principle be checked against the passage that supports it (Niu et al., 2024).

2.4 Use-Cases of RAG

RAG systems are in use across many domains: question-answering over internal document collections (Cheerla, 2025), customer support systems that must answer from current product or policy documentation, coding assistants that retrieve from a specific codebase instead of relying on general programming knowledge, legal question-answering over case law and contract repositories (Pipitone and Alami, 2024), and clinical decision support that grounds its answers in current medical literature and institutional protocols (He et al., 2025).

2.5 Advanced RAG Architectures

The architecture described so far retrieves once per query and generates once from what was retrieved. Recent work relaxes that procedure, and the resulting variants can be grouped by which stage of the pipeline they modify. None of these variants is used here, because this thesis studies the standard architecture as a baseline.

Conversational RAG. Retaining prior turns of a dialogue as a state, such that a follow-up question can be answered.

Pre-retrieval optimisation. Rewriting or expanding the query before it reaches the retriever. Hypothetical Document Embeddings generate a hypothetical answer to the query and embed that instead of the query itself, based on the rationale that an answer lies closer in embedding space to the passage containing it than a question does (Gao et al., 2023a).

Routing and adaptive retrieval. Deciding where and how much to retrieve. Adaptive-RAG varies the depth of retrieval with the difficulty of the query, answering simple questions directly and reserving multi-step retrieval for those that need it (Jeong et al., 2024).

Self-correction and reflection. Evaluating retrieved evidence before or during generation. Corrective RAG grades each retrieved document, decomposes the ones it keeps into smaller knowledge strips, discards the strips that are not relevant, and falls back to a wider search when retrieval fails (Yan et al., 2024). Self-RAG instead trains the model to decide for itself when to retrieve and to critique its own draft output, so that retrieval and generation intertwine (Asai et al., 2024).

Agentic RAG. Treating retrieval as one action among several available to an agent. A meta-agent delegates to document-specific or tool-specific agents and combines their results, which turns a fixed pipeline into a workflow whose steps are chosen at run time (Singh et al., 2025).

Understanding the standard architecture formalised in this thesis remains a prerequisite for understanding any of them. Each variant changes the retrieval function R , the generation function G , or the number of times they are applied, but all of them are still built from a retrieval step and a generation step, and all of them still produce a distribution over outputs.

This thesis therefore implements the simplest architecture and none of the variants above. The reason is methodological. The objective is to estimate the causal effect of the system prompt on the output, which Chapter 4 formalises as the intervention $\text{do}(S = S_j)$. That effect is identifiable only as long as the system prompt reaches the answer through a single path. Each of the advanced variants breaks that condition.

3 Literature Review

Chapter 2 introduced the components of a RAG system. This chapter reviews three areas of literature relevant to studying how the system prompt affects such a system.

3.1 Prompt Sensitivity and Instruction-Following in Language Models

The first branch of literature studies language models with no retrieval component. Cheng and Mastropaolo (2026) vary the system prompt given to instruction-tuned models for code generation, crossing prompt specificity and prompting strategy with model, programming language and decoding settings. They report that the system prompt influences the quality of generated code independently of the task description. Theirs is the closest published design to the one used here, and the present experiment carries it from code generation into a retrieval-augmented setting, where the model is also given evidence to work from. Mu et al. (2025) examine how reliably models follow a system prompt when the user turn contains instructions conflicting with it, and report that the system prompt does not always retain succeed in that competition. They establish the system prompt as a channel with measurable influence over behaviour, and it shows that the influence

is not guaranteed. Zheng et al. (2024) assign personas through the system prompt across a large set of roles and find that doing so does not reliably improve accuracy over a system prompt with no persona.

A larger body of research varies the user-side prompt. Lu et al. (2024) compare prompting techniques for in-context learning and measure sensitivity through gradient-based saliency scores. They report that sensitivity correlates negatively with accuracy. Zhuo et al. (2024) propose ProSA, which scores instance-level variation in the task instruction against decoding confidence, and report that larger models are more robust to such variation and that few-shot examples reduce it. Salinas and Morstatter (2024) perturb task prompts across text classification tasks (adding a trailing space, requesting the output in XML, or prefixing a jailbreak) and find that perturbations of this size change the labels the model assigns. Sclar et al. (2024) vary only the formatting of few-shot task templates (separators, casing and spacing, none of which carries propositional content) and report differences of up to 76 accuracy points between formats a reader would judge equivalent. Hua et al. (2025) run twelve task templates across several benchmarks and argue that much of the measured sensitivity is produced by the evaluation method (log-likelihood scoring and rigid answer matching). Qin et al. (2026) decompose a model’s output score into interactions between input variables and observe that small changes to the prompt destabilise those interactions even where the final output is unchanged.

Two further studies fall outside both branches of the literature. Ruiz and Cardona-Betancourt (2026) state a rule in the instruction given to a model and then shift the contingencies that rule was written for. Ying et al. (2024) study conflict between a model’s parametric memory and the evidence supplied in its context, and classify models

as intuitive or dependent by which source they favour.

Two conclusions follow. The first is that the system prompt is a behavioural control variable whose effect can be isolated and measured given a properly controlled setup, which the three system-prompt studies support directly (Cheng and Mastropaolo, 2026; Mu et al., 2025; Zheng et al., 2024). The second is that instructions are sensitive to changes that preserve their meaning, whether lexical (Lu et al., 2024; Zhuo et al., 2024; Salinas and Morstatter, 2024), purely formatting-level (Sclar et al., 2024), or not visible in the output at all (Qin et al., 2026), and that the apparent size of such an effect depends on how it is measured (Hua et al., 2025). This second conclusion is established for the user-side prompt, and its transfer to the system prompt is an assumption instead of a result. The assumption is reasonable, since both texts reach the same decoder as tokens in the same context window, but it is untested.

3.2 Retrieval-Augmented Generation: Architecture, Evaluation, and Failure Modes

Another body of literature concerns retrieval-augmented generation itself. Lewis et al. (2020) introduce RAG, combining a retriever with a pretrained sequence-to-sequence generator, and show that conditioning generation on retrieved evidence outperforms purely parametric models on knowledge-intensive tasks. Their probabilistic formulation, in which retrieval is a distribution over documents and the retrieved

document is marginalised out, is the direct basis for the model formalised in Chapter 4. Gao et al. (2023c) survey the development of RAG systems through Naive, Advanced and Modular paradigms. Their taxonomy supports treating retrieval and generation as separable functions whose interaction with the system prompt can be studied independently, as this thesis does. Two later developments extend this taxonomy in directions this thesis deliberately does not follow. Edge et al. (2024) restructure the retrieval side into a knowledge graph, grouped into thematic segments and pre-summarised by an LLM, so that a query is answered by traversing entities and summaries. Singh et al. (2025) and Jin et al. (2025) restructure the generation side into an agent loop, in which the model itself decides when and what to retrieve, calling R repeatedly and adaptively; Jin et al. (2025) train this behaviour directly with reinforcement learning and report improvements over standard RAG baselines. Both directions relax exactly the components Chapter 4 holds fixed: R in the first case, and the single-call structure of G in the second. For that reason this thesis retains Gao et al.’s simplest, Naive-RAG form, so that any change in the outcomes measured here can be attributed to the system prompt rather than to a graph-structured retriever or an agentic generation loop.

Es et al. (2023) propose Ragas, a reference-free evaluation framework for RAG pipelines built around three metrics that need no ground-truth human annotation: faithfulness, answer relevance and context relevance. This thesis adopts faithfulness and answer relevance. The third, context relevance, is a retrieval-side measure and is excluded for the reasons given in Section 5.7. Their framework also motivates evaluating the effect of the system prompt independently of retrieval quality. Chen et al. (2024) construct the Retrieval-Augmented Generation

Benchmark (RGB) and show that current language models tolerate a moderate amount of retrieval noise. Two of their four test environments shape the design of this thesis. Niu et al. (2024) introduce RAGTruth, a manually annotated corpus of word-level hallucinations produced by RAG systems, and benchmark hallucination-detection methods across multiple language models.

3.3 Synthesis and Research Gap

The first branch shows two things: that the system prompt is a controllable way of shaping language model behaviour, and that instructions are sensitive to rewording that leaves their meaning unchanged. It establishes the controllability result for the system prompt, the sensitivity result mostly for the user-side prompt, and both outside any retrieval-augmented setting. The second branch establishes the architecture, evaluation vocabulary, and known failure modes of RAG systems.

This thesis treats the system prompt as a controlled experimental variable inside a RAG pipeline, holding the query, knowledge base, retrieval mechanism, and underlying language model fixed, so that any change in the system’s behaviour and outcomes can be traced back to the prompt itself. Every condition varies only the wording of the instructions and leaves what they require untouched.

Chapter 4 builds the formal model that makes this control precise.

4 Formal Model of a RAG System

Chapter 3 identified the research gap: prompt sensitivity is studied outside retrieval, and RAG evaluation does not treat the system prompt as a controlled variable. Closing that gap requires a model of the system precise enough to isolate the system prompt. This chapter builds that model on the probabilistic formulation of RAG given by Lewis et al. (2020), and extends it by making the system prompt an explicit variable of the generation process.

4.1 Modelling Objective

The formalisation states the probabilistic relationship between the user query, the retrieved documents and the generated output. It also lets the system prompt be treated as an experimental intervention whose effect can be measured while the query, the knowledge base, the retriever and the language model are held fixed.

The model can be read as a chain,

$$(q, D) \longrightarrow p_{\eta}(z \mid q, D) \longrightarrow p_{\theta}(y_i \mid q, z, y_{1:i-1}, S) \longrightarrow p_{\text{RAG}}(y \mid q, D, S),$$

in which the system prompt S enters at the generation stage. The research question concerns that final distribution: what changes in

$p_{\text{RAG}}(y \mid q, D, S)$ when S is changed?

4.2 Probabilistic Retrieval

Retrieval is represented as a probability distribution over documents:

$$p_{\eta}(z \mid q, D). \quad (4.1)$$

Equation (4.1) is the probability the retriever assigns to document z , given the query q and the knowledge base D . Writing retrieval this way means the retriever never has to commit to one document being the correct one. A knowledge base may hold far more documents than can be evaluated in practice. The formulation therefore keeps only the K documents with the highest retrieval probability,

$$\mathcal{Z}_K(q, D) = \underset{z \in D}{\text{TopK}} \ p_{\eta}(z \mid q, D), \quad (4.2)$$

and the retrieved context handed to the generator is the resulting collection,

$$C = \{z_1, \dots, z_K\}, \quad z_k \in \mathcal{Z}_K(q, D). \quad (4.3)$$

The distinction between z and C in Equation (4.3) matters later.

4.3 Prompt-Conditioned Generation

Lewis et al. (2020) write the generator at the level of a single token as $p_{\theta}(y_i \mid q, z, y_{1:i-1})$. It is the probability of the next token given the

query, one retrieved document, and the tokens already produced. This thesis adds the system prompt as a further conditioning variable:

$$p_{\theta}(y_i \mid q, z, y_{1:i-1}, S). \quad (4.4)$$

Equation (4.4) is the point at which this thesis departs from the original formulation. The generator is the same model with the same parameters θ ; the system prompt is now written into the conditioning information.

Because generation is autoregressive, the probability of a complete output sequence follows from the token probabilities by multiplication:

$$p_{\theta}(y \mid q, z, S) = \prod_{i=1}^L p_{\theta}(y_i \mid q, z, y_{1:i-1}, S). \quad (4.5)$$

4.4 Marginalizing over Retrieved Documents

Equation (4.5) gives the probability of an output under document z . Since the system does not know which retrieved document supports the answer, the model averages over the chosen documents, weighting each by its retrieval probability. Lewis et al. (2020) give two ways of doing this.

4.4.1 RAG-Sequence

RAG-Sequence assumes that a single retrieved document is responsible for the whole answer. The probability of an output is the weighted

average, over documents, of the probability that document would produce that entire sequence:

$$p_{\text{RAG-Sequence}}(y \mid q, D, S) \approx \sum_{z \in \mathcal{Z}_K(q, D)} p_{\eta}(z \mid q, D) \prod_{i=1}^L p_{\theta}(y_i \mid q, z, y_{1:i-1}, S). \quad (4.6)$$

4.4.2 RAG-Token

RAG-Token makes the weaker assumption that different parts of an answer may come from different documents. The average over documents is taken separately at every token:

$$p_{\text{RAG-Token}}(y \mid q, D, S) \approx \prod_{i=1}^L \sum_{z \in \mathcal{Z}_K(q, D)} p_{\eta}(z \mid q, D) p_{\theta}(y_i \mid q, z, y_{1:i-1}, S). \quad (4.7)$$

The distinction does not matter for the research question, both are written here as $p_{\text{RAG}}(y \mid q, D, S)$.

4.5 The System Prompt: Requirements and Wording

The system prompt enters Equation (4.4) as a single symbol. It is a text with internal structure, and the experiment varies that structure deliberately. A prompt states a set of requirements and states them in a particular wording. Writing the two separately gives

$$S = \rho(\Sigma, \varphi), \quad \Sigma = \{\sigma_1, \dots, \sigma_m\}, \quad (4.8)$$

this formulation separates what a prompt asks for from the wording. Two prompts built from the same Σ ask for the same thing. Two built from the same φ say things the same way.

Three requirement set are used. Each was taken from a prompt published with a retrieval-augmented system and reproduced in Section 5.3. Σ_A is the response-generation prompt of Friel et al. (2024), which states a single requirement; Σ_B is the VANILLA instruction of Gao et al. (2023b), which states about ten clauses falling into four independently checkable families; and Σ_C is the instruction of Mao et al. (2024), which states three.

The second factor is the wording. Four wordings are written for each requirement set, holding Σ fixed and changing only φ . The dimensions along which the wording is varied are taken from three prior studies. Wahle et al. (2024) vary prompts by linguistically defined paraphrase type, and group those types into families, of which morphology, syntax and lexicon apply to a standing instruction. Zhan et al. (2024) look at the lexical dimension by substituting single semantically similar words, and report that models are over-sensitive to variations imperceptible to a reader. Kostić et al. (2026) separate a lexical pipeline built on synonym substitution from a syntactic one built on dependency parsing, and find that the two behave differently. Lexical perturbation degrades performance consistently, syntactic perturbation does not.

This thesis follows their approach of varying the system prompt along linguistically defined, meaning-preserving dimensions one at a time. Writing $S_{b,j}$ for the j -th wording of baseline b :

$S_{b,0} = \rho(\Sigma_b, \varphi_0)$, **canonical**. The published form, unchanged. The

reference the other three are measured against.

$S_{b,1} = \rho(\Sigma_b, \varphi_1)$, **lexical**. Synonym substitution with clause order and voice held fixed.

$S_{b,2} = \rho(\Sigma_b, \varphi_2)$, **syntactic**. Clause reordering and active to passive, with the vocabulary held as far as English allows.

$S_{b,3} = \rho(\Sigma_b, \varphi_3)$, **format**. The same wording laid out as a list rather than as prose.

Baseline *A* states the fewest requirements and is short enough to give in full as an example. The four conditions read:

A0 Use the following pieces of context to answer the question.

A1 Employ the following excerpts of material to address the query.

A2 The question is to be answered using the context pieces below.

A3 CONTEXT: the following pieces
TASK: use them to answer the question

Listing 4.1: The four conditions of baseline *A* from Friel et al. (2024).

The variants for baselines *B* and *C* are written on the same principle and are longer in proportion to their requirement sets. Within each baseline the four conditions are held to within one tenth of the canonical form in length.

4.6 The System Prompt as an Intervention

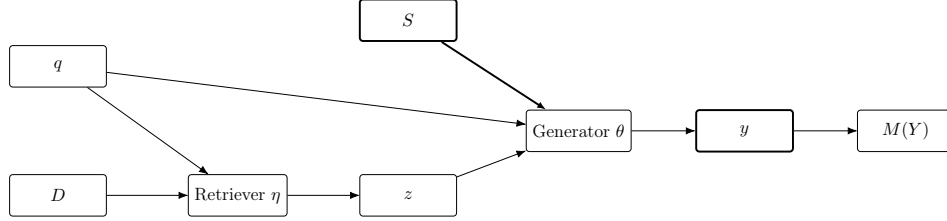


Figure 4.1: The system prompt S and the output y are drawn in bold. S enters the generator and has no arrow into the retriever, which is what allows a change in y to be attributed to a change in S .

For the effect of the prompt to be interpretable, the following are held constant across conditions. Each query is used unchanged in every condition it is run in, so that every comparison is made within a query rather than between queries, and the knowledge base is fixed at $D = D_0$. The retriever and its parameters are fixed at $\eta = \eta_0$, and the language model at $\theta = \theta_0$.

What is deliberately not held fixed is the generator’s output distribution. Although θ stays fixed, the conditional distribution it defines may differ between S_0 and S_j : that difference is the effect the experiment is built to measure. Decoding is also stochastic, so the same configuration can produce different outputs from one run to the next. The term ε captures that variation:

$$Y = G(q, z, S; \theta, \varepsilon), \quad (4.9)$$

which is why each condition is repeated several times.

4.7 The Causal Effect

Setting the system prompt to a fixed value is an intervention, written $\text{do}(S = S_j)$ following Pearl (2009). The notation separates *setting* a variable from merely *observing* it. For a query q under prompt condition S_j , the output of the system is the random variable

$$Y \sim p_{\text{RAG}}(y \mid q, D, S_j). \quad (4.10)$$

For one query the effect of the prompt is the difference in the expected value of the measured property $M(Y)$ of Equation (4.10) between the two conditions. That difference is not, however, the quantity of interest. A prompt that improves the answer to one question and degrades the answer to the next has established no effect at all, and a difference observed on a single input cannot be separated from the particular difficulty, phrasing or subject matter of that input. The research question asks about the behaviour of the system over the queries it will actually meet, so the effect has to be defined as an average over the distribution from which queries are drawn. Writing P_Q for that distribution over the query space $\mathcal{Q}$, the causal effect of prompt condition S_j relative to the baseline S_0 is

$$\tau_j = \mathbb{E}_{q \sim P_Q} \left[\mathbb{E}_{Y \sim p_{\text{RAG}}(\cdot \mid q, D, S_j)} [M(Y)] - \mathbb{E}_{Y \sim p_{\text{RAG}}(\cdot \mid q, D, S_0)} [M(Y)] \right], \quad (4.11)$$

which connects the probability model of Section 4.4 directly to the quantity Chapter 5 estimates.

Equation (4.11) is an average treatment effect in the standard sense: the query plays the role of the unit, the prompt condition the role of the treatment, and τ_j the mean over units of the difference between

the two treatments. Two features of how it is written matter. The difference is taken inside the outer expectation. That is what makes each query its own control, and it matches the paired design of Section 4.6. By linearity of expectation the two forms agree, but only the form given here records the pairing. The expectations are also nested rather than pooled: the inner expectation averages over the randomness of retrieval and decoding at a fixed query, while the outer averages over the queries themselves.

The distribution P_Q is not known and cannot be written down. Chapter 5 replaces it with the empirical distribution of a query set. That is the point at which the choice of data becomes a claim about generality, and it is why Section 4.8 draws the queries from established question-answer datasets.

The choice of M is left open here. Section 4.9 describes what that property has to capture, and Section 5.7 fixes it.

4.8 Grounding the Knowledge Base in Established Dataset

D and the query set are instantiated from an established question-answer dataset. The corpus is public, versioned and split in a documented way, and therefore a third party could repeat the experiment. Because published results for a wide range of retrieval and generation systems already exist on these datasets, the effect estimated in Chapter 5 can be read against a background of prior work.

The collection is built from queries sampled from Bing search logs, with candidate passages retrieved for each query and free-form an-

swers written by human judges (Nguyen et al., 2016). MS MARCO provides the queries q , the documents that populate D , and, through its annotations, both the reference answers A_q and the fact set against which an output can be judged.

What it does not provide is a definition of a good answer. An annotation records which answer is correct; it says nothing about whether an answer is grounded in the evidence actually retrieved, whether it addresses the question directly, or whether it obeys the requirements the system prompt states. Those properties are the object of this thesis, and defining them is the subject of the next section.

4.9 What Counts as a Good Answer

Quality in a retrieval-augmented system is not one property but several, and they can fail independently of one another. The first is groundedness. An answer should rest on the documents the retriever actually supplied. The second is relevance. An answer should address the question that was asked and address it directly. The third is correctness. An answer should agree with what is in fact the case, which is a different demand from the first and is treated separately here for the reason given below. The fourth is obedience. Whatever the system prompt asks for should be visible in the output itself, whether in length, in form, or in the handling of missing evidence, since a prompt that is not followed cannot be said to have any effect on quality at all.

Groundedness and correctness are kept apart deliberately, because collapsing them would conceal the failure this thesis is concerned with. The retrieved context is neither complete nor guaranteed to be free of

error. An answer can therefore be correct while ignoring the evidence entirely, drawing instead on what the generator already holds in its parameters, and it can use the evidence faithfully and still be wrong because the evidence was wrong. A system that scored well on correctness alone would reveal nothing about whether retrieval was used at all, which is precisely the behaviour a system prompt is supposed to govern.

All four are properties of the text the generator produces. Each of the four is estimated by a measure taken from published work. Groundedness by faithfulness, which asks whether every claim can be traced to a retrieved passage; relevance by answer relevance, which asks whether the response addresses the question put to it; correctness by agreement with the reference answer recorded in the data; and obedience by whether the output satisfies every instruction the prompt states.

Chapter 5 states the set of acceptable answers precisely and fixes the measures that estimate the four properties.

5 Experimental Setup

Chapter 4 built a model of the RAG system and defined the effect of a prompt condition as the average treatment effect τ_j of Equation (4.11). This chapter turns that model into a concrete experiment. All experiments are implemented in Python.

5.1 Research Question and Hypothesis

Chapter 1 asked to what extent systematic changes in the system prompt influence the behaviour and outcomes of a RAG system when the user input, the knowledge base, the retrieval mechanism and the language model are held fixed. The experiment puts that question in the form it can be tested in: *does a RAG system behave differently when the structure of its system prompt is changed while the meaning of the prompt stays the same?*

Structure means how the prompt is written: which words are used, how the sentences are built, and how the text is laid out on the page. Meaning means the set of requirements Σ that the prompt states, in the sense of Equation (4.8). Every condition of the experiment keeps Σ fixed and changes only the wording φ . If the system responded only to what a prompt requires and not to how it is written, all wordings of one prompt would produce the same distribution of answers. Whether they do is the empirical question. Stated as hypotheses:

H_0 . System prompts that state the same requirements in different

wordings produce the same distribution of answer quality: $\tau_j = 0$ for every rewording j , in every baseline, on every measure and at both evidence levels.

H_1 . At least one rewording changes answer quality: $\tau_j \neq 0$ for some rewording in some baseline on some measure at some evidence level.

5.2 Overview of the Experiment

The system under test is the simple RAG architecture of Chapter 2. It is used for the reason given in Section 2.5: in this architecture the system prompt reaches the answer along one path, through the generator, so its effect can be identified. A query is embedded, the K most similar passages are retrieved from a vector index, and a language model produces the answer from the system prompt, the passages and the query. Table 5.1 lists the components of that system and the value each one is fixed at.

The experiment has three stages, of which the first two collect data. In the first stage, retrieval is run once for every query and its result is written to disk. In the second stage, every prompt condition is applied to every query, reading the stored retrieval result, and the answers are written to disk. In the third stage, the answers are scored and the prompt effect is estimated.

Table 5.1: The components of the system under test and their fixed values.

Component	Fixed value
Knowledge base D	MS MARCO v2.1 validation passages, $N = 122,678$
Chunking χ	one passage per chunk, as published
Embedding model	<code>all-mpnet-base-v2</code> , mean pooling, L2-normalised
Vector index	FAISS flat inner-product index, exact search
Retrieval depth K	10
Generator θ	<code>gpt-4o-mini</code> via the chat completions API
Decoding	temperature 1.0 (the API default), at most 256 new tokens
Judge	<code>gpt-4.1-mini</code> , temperature 0, fixed prompts
Query set	100 MS MARCO queries, seed 20260904
Repetitions n	3 per cell

5.3 System-Prompt Conditions

Baseline A is the response-generation prompt of Friel et al. (2024), reproduced in Listing 5.1. It states one requirement: answer from the supplied context.

```
Use the following pieces of context to answer the question.
{documents}
Question: {question}
```

Listing 5.1: Baseline A : the response-generation prompt of Friel et al. (2024).

Baseline B is the VANILLA instruction of Gao et al. (2023b). It constrains accuracy, engagement, concision and register, it fixes the syntax and the density of citations, and it warns that some of the supplied results may be irrelevant.

```
Write an accurate, engaging, and concise answer for the given
question using only the provided search results (some of
which
```

```

might be irrelevant) and cite them properly. Use an unbiased
    and
journalistic tone. Always cite for any factual claim. When
    citing
several search results, use [1][2][3]. Cite at least one
    document
and at most three documents in each sentence. If multiple
    documents
support the sentence, only cite a minimum sufficient subset
    of the
documents.

```

Listing 5.2: Baseline *B*: the VANILLA instruction of Gao et al. (2023b), reproduced from their Table 23.

Baseline *C* is the instruction of Mao et al. (2024). Its authors describe it as three parts: the first directs the model to the retrieved passages, the second asks it to read the question carefully, and the third asks for an answer together with a justification.

```

Refer to the passage below and answer the following question.
Make sure you fully understand the meaning of the question
    and
passages. Then give the answer and explain why you choose
    this
answer.
Passages: {passages}
Question: {question}

```

Listing 5.3: Baseline *C*: the comprehensive instruction of Mao et al. (2024).

The published prompts contain placeholders for the documents and the question. In this experiment the instruction text alone forms the system prompt *S* and is sent in the system role of the chat interface.

The passages and the question are sent in the user role, in the fixed template of Listing 5.4. The template is the same in every condition, so the only text that differs between conditions is S . Passages are numbered from one, so that a citation instruction has something to refer to and so that the position of a passage can be recovered from the answer.

```

Passages:
[1] <first passage>
[2] <second passage>
...

Question: <the query>
Answer:

```

Listing 5.4: The user turn. It is identical in every condition; only the system turn S changes. With no passages, the line after `Passages:` reads `(none)`.

Each baseline is written in four wordings. The first is the canonical form above. The other three each change one aspect of the wording and leave everything else as it is. The dimensions follow the paraphrase types of Wahle et al. (2024), Zhan et al. (2024) and Kostić et al. (2026), as explained in Section 4.5. Table 5.2 states what each wording changes and the rule it follows.

A rewording that changed how strongly an instruction is stated, for example by turning *always cite* into *please try to cite*, would change what the prompt requires. Such a change is a change of Σ , and a difference in behaviour caused by it could not be read as an effect of structure. No condition of this experiment makes a change of that kind.

Table 5.2: The four wordings. Every wording of a baseline states the same requirement set Σ_b .

j	Wording	What changes, and the rule kept
0	canonical	Nothing. The published form.
1	lexical	Words are replaced by synonyms. Clause order and voice are kept.
2	syntactic	Clauses are reordered and active voice becomes passive. The vocabulary is kept as far as English allows. Obligation is carried uniformly by <i>is to be</i> , so that the change of voice does not change how strongly something is required.
3	format	The same wording is laid out as a list with headings instead of as prose. List items are bare noun phrases, so that no obligation is added or removed.

The four wordings of baseline A are given in Listing 4.1 in Chapter 4. Listings 5.5 and 5.6 give the three rewordings of baselines B and C ; their canonical forms are Listings 5.2 and 5.3 above.

```

B1  Compose a precise, compelling, and succinct reply for the
    posed
    query drawing only on the supplied search results (a
    number of
    which may be immaterial) and reference them correctly.
    Adopt an
    impartial and journalistic register. Invariably reference
    every
    factual assertion. When referencing several search
    results,
    employ [1][2][3]. Reference at minimum one document and
    at
    maximum three documents per sentence. If several
    documents
    corroborate the sentence, only reference a minimal
    adequate
    subset.

```

- B2 For the given question an accurate, engaging, and concise answer
is to be written from the provided search results alone,
some
of which might be irrelevant, and cited properly. An
unbiased,
journalistic tone is to be used. A citation is always
given for
any factual claim. Where several results are cited,
[1] [2] [3]
is used. In each sentence at least one and at most three
documents are cited. Where multiple documents support a
sentence, only a minimum sufficient subset is cited.
- B3 ANSWER
- accurate, engaging, and concise, for the given question
 - source: only the provided search results
 - note: some of the provided results might be irrelevant
- CITATIONS
- cite the search results properly
 - always cite for any factual claim
 - when citing several search results: [1] [2] [3]
 - in each sentence: at least one document, at most three documents
 - if multiple documents support the sentence: only a minimum
sufficient subset
- TONE
- unbiased and journalistic

Listing 5.5: The lexical ($B1$), syntactic ($B2$) and format ($B3$) wordings of baseline B .

```

C1  Consult the excerpt below and respond to the following
    query.

    Ensure you completely grasp the sense of the query and
    excerpts.

    Then supply the response and justify why you select this
    response.

C2  The following question is to be answered from the passage
    below.

    The meaning of the question and passages is first
    understood.

    The answer is then given, with an explanation of why it
    is
    chosen.

C3  SOURCE: the passage below, and nothing else
    READ:  fully understand the meaning of the question and
    passages
    ANSWER: give the answer to the following question
    EXPLAIN: say why you choose this answer

```

Listing 5.6: The lexical ($C1$), syntactic ($C2$) and format ($C3$) wordings of baseline C .

Prompt length is held nearly constant within a baseline, so that a difference between two wordings cannot be an effect of how much text the prompt contains. Table 5.3 gives the word counts. No wording departs from its canonical form by more than one tenth, and the code refuses to run if an edit to a prompt breaks that bound.

Table 5.3: Word count of every condition, with the deviation from the canonical form of its baseline.

Baseline	canonical	lexical	syntactic	format
<i>A</i>	10	10 ($\pm 0\%$)	11 ($+10\%$)	11 ($+10\%$)
<i>B</i>	74	71 (-4%)	80 ($+8\%$)	77 ($+4\%$)
<i>C</i>	33	32 (-3%)	34 ($+3\%$)	32 (-3%)

Twelve conditions result, three baselines in four wordings each. They are written $S_{b,j}$ with $b \in \{A, B, C\}$ and $j \in \{0, 1, 2, 3\}$. Every comparison is made within a baseline. Comparing $S_{B,1}$ with $S_{A,0}$ would mix a change of wording with a change of what is asked for, so each rewording is compared only with the canonical form of its own baseline.

5.4 Variables and Controlled Factors

Independent variable. The wording j of the system prompt, with four levels, nested within the baseline b . Changing j changes the structure of the prompt and nothing else. The requirement set Σ_b is the same at every level of j .

Dependent variable. The quality of the answer, as scored by the measures of Section 5.7: faithfulness, answer relevance, correctness and instruction adherence, with correctness measured twice, by ROUGE-L and by BLEU-1. Each of the five is used in turn as the measured property M of Equation (4.11), so the effect of a rewording is estimated five times, once per measure. The five are reported separately and are not combined into a single score, for the reason given in Section 5.7.3.

Controlled factor. The context C handed to the generator is set directly, as the intervention $\text{do}(C = c_{ik})$ of Chapter 4, at two levels:

c^{gold} . Only the passages the annotation marks as supporting the answer. The evidence needed for the answer is certainly present, so a failure here belongs to the generator rather than to retrieval. Every query is usable at this level, which makes it the more sensitive of the two.

c^{ret} . The K passages the fixed retriever actually returns. This is the system as it would be deployed.

The retrieval depth is not a second factor. It is held constant at $K = 10$. That depth admits more of the annotated evidence than a shallower one, as Table 5.4 shows, so more queries are usable at the retrieved level and the comparison there rests on a larger sample. Ten passages also give the prompt a longer and noisier context to govern than fewer would.

Held constant. Everything in Table 5.1. The generator is one model behind one API, with a fixed temperature and a fixed output limit. Queries and passages are encoded by one embedding model, applied to both sides in the manner of the dense retriever of Karpukhin et al. (2020). The index, built with the library of Johnson et al. (2021), is constructed once. Each passage is one chunk. Because the embedding model, the index, the chunking and the depth are all fixed, the set $\mathcal{Z}_K(q, D)$ of Equation (4.2) depends on the query alone and never on the system prompt. The context is held identical in a stronger sense than that: the set of passages and their order are the same in every condition, since both are determined by the query, the evidence level and the repetition only.

Covariates. Two properties of each query are recorded and can be conditioned on in the analysis: the category (direct or multi-hop) and MS MARCO’s intent type. Both are defined in Section 5.5.

The design. An observation is one answer, written

$$y_{ijkr} \sim p_{\text{RAG}}(y \mid q_i, \text{do}(C = c_{ik}), \text{do}(S = S_j)), \quad r = 1, \dots, n, \quad (5.1)$$

for query i , prompt condition j , evidence level k and repetition r . The twelve prompt conditions and the two evidence levels are fully crossed, which gives 24 cells. Every cell is run for all 100 queries and repeated $n = 3$ times, which is 7,200 generations before the retrieval gate of Section 5.6 removes any. Within a cell the only remaining source of variation is the decoding randomness ε of Equation (4.9). Each repetition is an independent draw from the same distribution. Three repetitions is fewer than the five Zhuo et al. (2024) use for the same purpose, and the cost of that choice is stated in Section 6.4.

5.5 Knowledge Base and Query Set

Following Section 4.8, both the knowledge base and the query set come from MS MARCO (Nguyen et al., 2016), read from the Hugging Face export of version 2.1. The validation split is used. It holds 101,093 queries, of which 12,467 carry a non-empty `wellFormedAnswers` field. Only those rows are used, because the well-formed answers are the references against which correctness is measured, and a query without one cannot be scored.

The knowledge base. The corpus holds every passage attached to any of the 12,467 rows. MS MARCO ships about ten candidate passages per query, retrieved by Bing and then judged by annotators, so the corpus contains the gold evidence for every query. Each passage is one chunk and is used at the length at which it was published. The median passage has about 50 words and three sentences.

The query set. $N_Q = 100$ queries are sampled from the same 12,467 rows. They fall into two categories. A row is *direct* when exactly one of its passages is marked `is_selected`, and *multi-hop* when two or more are. Fifty queries are drawn from each category.

Q_{direct} . The answer is carried by a single passage.

$Q_{\text{multi-hop}}$. The human-written answer draws on two or more passages.

Within each category the draw is balanced across MS MARCO’s five intent types, DESCRIPTION, ENTITY, LOCATION, NUMERIC and PERSON, ten queries per type. The natural distribution is uneven: on the well-formed rows DESCRIPTION makes up 44% and PERSON 7%. Balancing makes the type usable as a covariate. The multi-hop half is drawn from a much smaller pool than the direct half. Of the 12,467 well-formed rows, 12,062 are direct and only 386 are multi-hop.

5.6 Experimental Procedure

5.6.1 Stage 1: Retrieval and the Retrieval Gate

Queries and passages are encoded with `all-mpnet-base-v2`, using mean pooling over the last layer and L2 normalisation, so that the

inner product of two vectors is their cosine similarity. The index is a flat inner-product index: every query is compared against every passage, with no approximation. At 122,678 vectors of 768 dimensions the index occupies 0.38 GB and fits in memory, and exactness matters more than speed here, because an approximate index would make retrieval depend on how the index was built instead of on the query alone.

Retrieval is run once, at $K = 10$, for all 100 queries. The identifiers, scores and ranks of the returned passages are written to disk.

The retrieval stage is also where the evidence is checked. A query passes the *retrieval gate* when every passage its annotation marks as supporting appears among the $K = 10$ returned. For a direct query that is one passage, whereas for a multi-hop query it is every passage the answer draws on. Table 5.4 reports the result. It is a property of the fixed retriever and the corpus, not of any prompt condition, and it is reported before the results because it sets the denominator of the retrieved level.

Table 5.4: The retrieval gate at the depth used, $K = 10$. A query passes when all of its gold passages are among the ten returned.

K	Passed	direct	multi-hop
10 (used)	88 / 100	45 / 50	43 / 50
5	74 / 100	41 / 50	33 / 50

Queries that fail the gate are excluded from the retrieved level. The measures of Section 5.7 are not defined on evidence that was never supplied. When the context lacks the passage carrying the answer, a correct answer can only have come from the generator’s memory, and an incorrect answer cannot be told apart from a faithful reading of the other passages. Scoring such a cell would measure the retriever and

report the result as an effect of the prompt.

5.6.2 Stage 2: Generation

For every query, every cell of the design is run $n = 3$ times. The cells are visited in a random order per query, so that any drift in the API over the collection period cannot align with a condition. Every generation is an independent request. The context for a cell is built as follows.

- At c^{gold} , the passages are the query’s gold passages.
- At c^{ret} , the passages are the top K from the stored retrieval output.

The order of the passages within the context is shuffled, and the positions of the gold passages are recorded, because answer accuracy depends on where the relevant passage sits (Liu et al., 2024). The system prompt $S_{b,j}$ is sent in the system role and the passages and question in the user role, in the template of Listing 5.4. The generator is `gpt-4o-mini` at temperature 1.0 with at most 256 new tokens.

For each generation the following are written to disk: the query and its category and type; the condition, evidence level, depth and repetition; the identifiers and order of the passages in the context and the positions of the gold passages; the exact system prompt; the model identifier, the decoding settings and the backend fingerprint the API returned; the answer and its token count; and a timestamp.

5.6.3 The Pipeline

The whole procedure, in order:

1. Download the MS MARCO v2.1 validation split and keep the 12,467 rows with a well-formed answer.
2. Embed every passage attached to those rows and build the index: 122,678 passages.
3. Sample the query set: 50 direct and 50 multi-hop queries, ten per intent type, seed 20260904.
4. Retrieve the top 10 passages for every query and apply the retrieval gate. The top 5 are retrieved as well, so that the gate can be reported at that depth for comparison in Table 5.4, but no generation is run at that depth.
5. For every query, every one of the 24 cells and every one of the 3 repetitions, build the context, send $S_{b,j}$ and the user turn to the generator, and record the answer.
6. Score every answer with the five measures of Section 5.7.
7. Estimate the effect of each rewording in every cell, once per measure, with a confidence interval.

5.7 Evaluation

Many different texts can express the same answer, so the evaluation defines a set of acceptable answers. An answer belongs to that set when it satisfies four requirements at once:

Grounded. It asserts only what the supplied passages state, and introduces nothing from outside them.

Relevant. It addresses the question that was asked, and addresses it directly.

Correct. It agrees with the reference answer of the dataset annotation.

Obedient. It satisfies every requirement the system prompt states.

An answer that breaks none of the four is a good answer. Each requirement is estimated by one measure, except correctness, which is estimated by two. All of them are taken from published evaluation work. The first three are the measures that the survey of Yu et al. (2024) names as the evaluation targets of the generation component: faithfulness relates the response to the retrieved documents, answer relevance relates it to the query, and correctness relates it to the reference answer. Retrieval measures are excluded by construction, because the system prompt cannot change what is retrieved.

5.7.1 Five Measures for Four Properties

Faithfulness. Whether every claim in the answer can be traced to a supplied passage. It is scored reference-free, after Es et al. (2023), in two steps. First the judge (Section 5.7.2) breaks the answer into atomic factual claims. Then, for each claim, the judge decides whether the passages state it or directly include it. The breach count $n_1(y)$ is the number of unsupported claims. The reported rate is the supported fraction, since a raw count grows with answer length and prompts

change how much a model writes. This is the measure the experiment relies on. Every one of the twelve conditions states a grounding constraint, and faithfulness shows whether that constraint is followed in practice. Because the passages are identical across conditions, a change in faithfulness cannot come from a change in what the generator was given.

Answer relevance. Whether the answer addresses the question actually asked. A prompt can degrade an answer without making it false. Relevance is scored reference-free, following Es et al. (2023): the judge writes three questions that the answer would be a good answer to, and those are compared with the question actually asked by their mean cosine similarity in the space of the retrieval encoder, such that no second embedding space is introduced. A mean below 0.5 counts as a breach. Both evidence levels of this design supply passages, so an answer that declines to answer has declined in the presence of evidence and counts as a breach. A refusal is detected by a fixed list of patterns such as *cannot be answered* and *not enough information*.

Correctness. Whether the answer agrees with the reference answers A_q . It is measured with the two metrics MS MARCO uses to score its free-form answers (Nguyen et al., 2016), and that Lewis et al. (2020) report on that task: ROUGE-L (Lin, 2004) and BLEU-1 (Papineni et al., 2002). Both compare the answer with the references word by word. Before scoring, the text is lowercased and punctuation is turned into spaces, and citation markers such as [3] are removed from the answer, because baseline B asks for them and they would otherwise lower its score for following its prompt. ROUGE-L is the F-measure of the longest common subsequence of the answer and a reference: its

precision and its recall are each taken at their best over the references and combined with $\beta = 1.2$, which weights recall slightly above precision, as the MS MARCO scorer does. BLEU-1 is the share of the answer’s words that also occur in a reference, each word counted at most as often as it appears in any single reference, multiplied by a brevity penalty when the answer is shorter than the reference closest to it in length. The official scorer computes BLEU over the whole corpus; since the estimate averages a value per answer, the per-answer form is used here. Both scores lie between 0 and 1.

Instruction adherence. Whether the answer does what the prompt told it to. It is scored as prompt-level strict accuracy in the sense of Zhou et al. (2023): an answer is adherent only if it satisfies every checked requirement, so the score is binary. For baseline *B* the rule parses the citation markers and checks that every sentence carries between one and three of them, which is what the prompt asks for. For baseline *C* the rule checks that the answer contains a justification marker. The markers are the causal connectives *because*, *since* and *therefore*, the phrases *this is why* and *the reason*, and three phrases that point to a source: *as the passage* (or document, or text), *according to* and *based on*. The last group names where an answer came from rather than why it was chosen, so the rule is more forgiving than the requirement it stands for. Baseline *A* states only grounding, which faithfulness already measures, so its adherence is undefined by construction. The requirement set is the same for all four wordings of a baseline, so adherence is directly comparable across them.

5.7.2 The Judge

Faithfulness and answer relevance have no reference string. Therefore, they are scored by a second language model, as Es et al. (2023) and Saad-Falcon et al. (2024) do. The judge is `gpt-4.1-mini`, a different model from the generator, so that the generator never grades its own output. It is run at temperature 0 to ensure that the verdict depends on the answer and the passages and not on a random draw. Since this thesis assumes that language models are sensitive to their instructions, the judge cannot be assumed reliable. Two precautions are taken, following Zheng et al. (2023), who document position and verbosity biases in this setting. The judge’s own prompts are fixed constants. The judge is shown only the answer and the passages, never the condition label, the query identifier or the evidence level.

5.7.3 Estimating the Effect of a Rewording

The metric reported is the average treatment effect of Equation (4.11). Chapter 4 left the measured quality of the answer M as a placeholder. It is fixed here by the five measures in turn, so the effect of a rewording is estimated five times: once for faithfulness, once for answer relevance, once each for ROUGE-L and BLEU-1, and once for instruction adherence. Every measure is oriented so that a higher value is better, so a positive effect means the rewording improved that measure.

For one measure M , one evidence level and one baseline b , the estimate of the effect of wording j is

$$\hat{\tau}_j = \frac{1}{|I|} \sum_{i \in I} (\bar{M}_{ij} - \bar{M}_{i0}), \quad \bar{M}_{ij} = \frac{1}{n} \sum_{r=1}^n M(y_{ijr}), \quad (5.2)$$

where I is the set of queries in that cell and $\bar{M}_{ij}$ is the mean of the measure over the n repetitions of query i under condition $S_{b,j}$. Equation (5.2) is the sample version of Equation (4.11). The inner average takes the expectation over decoding, and the outer one takes it over queries.

6 Results and Discussion

Chapter 5 fixed the experiment. It comprises twelve system prompts, two evidence levels, a fixed retriever and generator, and five measures, correctness being measured twice. This chapter reports the effect of each rewording and states the verdict on the hypothesis of Section 5.1.

The run produced 6,768 answers. At the gold level all 100 queries are usable, which gives $100 \times 12 \times 3 = 3,600$ answers. At the retrieved level 88 of the 100 queries pass the gate of Section 5.6 at $K = 10$, which gives $88 \times 12 \times 3 = 3,168$. Every answer was scored on the five measures of Section 5.7, and the two judged measures took 30,636 calls to the judge model.

6.1 How the System Behaved Before Any Comparison

Table 6.1 gives the mean of each measure in each cell.

The measures differ sharply in how much room they leave a prompt to move. Faithfulness is between 0.912 and 0.976 everywhere. Relevance is stable, between 0.693 and 0.746. Adherence runs from 0.083 to 0.898, a spread of more than eighty points across conditions that ask for the same thing, and it is the only measure with room to move that far. ROUGE-L runs from 0.201 to 0.550 and BLEU-1 from 0.119 to 0.505. Both are highest under baseline *A*, whose answers are the

Table 6.1: Mean of each measure per condition. The gold level rests on 100 queries and the retrieved level on the 88 that pass the gate at $K = 10$. Correctness is given as ROUGE-L and BLEU-1. Adherence is undefined for baseline A, whose only requirement is grounding.

	Faithfulness	Relevance	ROUGE-L	BLEU-1	Adherence
<i>gold</i> , $ I = 100$					
A0	0.917	0.736	0.541	0.494	n/a
A1	0.922	0.730	0.519	0.463	n/a
A2	0.930	0.734	0.550	0.505	n/a
A3	0.930	0.741	0.539	0.492	n/a
B0	0.912	0.696	0.362	0.285	0.223
B1	0.942	0.705	0.407	0.332	0.447
B2	0.924	0.703	0.397	0.319	0.343
B3	0.931	0.703	0.403	0.327	0.297
C0	0.924	0.729	0.262	0.164	0.890
C1	0.919	0.738	0.272	0.174	0.447
C2	0.941	0.728	0.296	0.193	0.433
C3	0.939	0.731	0.322	0.214	0.817
<i>retrieved</i> , $K = 10$, $ I = 88$					
A0	0.968	0.737	0.436	0.377	n/a
A1	0.976	0.734	0.418	0.359	n/a
A2	0.974	0.732	0.443	0.383	n/a
A3	0.976	0.736	0.438	0.376	n/a
B0	0.962	0.693	0.233	0.157	0.083
B1	0.971	0.705	0.297	0.220	0.345
B2	0.957	0.696	0.267	0.192	0.193
B3	0.965	0.702	0.282	0.205	0.205
C0	0.963	0.734	0.201	0.119	0.898
C1	0.973	0.738	0.207	0.125	0.568
C2	0.973	0.746	0.229	0.141	0.473
C3	0.959	0.730	0.267	0.170	0.886

shortest. One pattern in the table runs against expectation. In every condition, faithfulness is higher at the retrieved level than at the gold level. The gold context contains only the passages annotated as necessary, so it is the easier setting for answering the question but the harder one for being fully supported: with fewer passages in front of it, more of what the model writes has no source in the context to rest on.

6.2 The Effect of Rewording

Using Equation (5.2), each rewording is compared with the canonical form of its own baseline. Every comparison is paired within a query.

6.2.1 Instruction Adherence

The largest effects appear on this measure. Table 6.2 gives all six estimates in both cells. It covers baselines *B* and *C* only, because baseline *A* asks for nothing except that the answer come from the supplied context, which faithfulness already measures. Hence, no checkable instruction is left for adherence and the measure is skipped for *A* instead of being scored as a pass.

Eleven of the twelve intervals exclude zero after correction. Five of the six rewordings do so in both cells, which means the same rewording produced the same change on two different sets of evidence. Only C3 fails at the retrieved level, and it is the smallest effect in the table.

Table 6.2: Effect of each rewording on instruction adherence, against the canonical form of its own baseline. A star marks an interval that excludes zero after the correction across the six rewordings of the adherence family at that evidence level.

Rewording	gold		retrieved, $K = 10$	
	$\hat{\tau}$	95% CI	$\hat{\tau}$	95% CI
B1 lexical	+0.223*	[+0.158, +0.289]	+0.261*	[+0.187, +0.336]
B2 syntactic	+0.120*	[+0.071, +0.169]	+0.110*	[+0.045, +0.175]
B3 format	+0.073*	[+0.026, +0.121]	+0.121*	[+0.058, +0.185]
C1 lexical	-0.443*	[-0.521, -0.366]	-0.330*	[-0.410, -0.249]
C2 syntactic	-0.457*	[-0.534, -0.379]	-0.424*	[-0.501, -0.348]
C3 format	-0.073*	[-0.142, -0.005]	-0.011	[-0.071, +0.049]

6.2.2 Faithfulness

One of the nine intervals at the gold level excludes zero after correction, namely B1, $\hat{\tau} = +0.031$ with a 95% interval of [+0.014, +0.047]. The same comparison at the retrieved level gives +0.009 with an interval that contains zero. A result that appears in one cell and not the other is weak evidence.

6.2.3 Answer Relevance

No interval excludes zero after correction in either cell. The largest estimate is +0.012. Relevance sits between 0.693 and 0.746 in every condition and the rewordings do not move it.

6.2.4 Correctness

Most rewordings moved correctness. Tables 6.3 and 6.4 give the estimates.

Every rewording of baseline B raises correctness, by 0.034 to 0.064.

Table 6.3: Effect of each rewording on correctness measured by ROUGE-L, against the canonical form of its own baseline. A star marks an interval that excludes zero after the correction across the nine rewordings at that evidence level.

Rewording	gold		retrieved, $K = 10$	
	$\hat{\tau}$	95% CI	$\hat{\tau}$	95% CI
A1 lexical	-0.022*	[-0.036, -0.008]	-0.018*	[-0.031, -0.005]
A2 syntactic	+0.009	[-0.002, +0.021]	+0.008	[-0.006, +0.022]
A3 format	-0.002	[-0.016, +0.012]	+0.002	[-0.007, +0.011]
B1 lexical	+0.045*	[+0.027, +0.062]	+0.064*	[+0.042, +0.087]
B2 syntactic	+0.034*	[+0.020, +0.049]	+0.034*	[+0.016, +0.051]
B3 format	+0.041*	[+0.027, +0.054]	+0.049*	[+0.028, +0.070]
C1 lexical	+0.010	[-0.000, +0.020]	+0.006	[-0.002, +0.013]
C2 syntactic	+0.034*	[+0.025, +0.043]	+0.027*	[+0.020, +0.034]
C3 format	+0.060*	[+0.047, +0.072]	+0.066*	[+0.054, +0.077]

Table 6.4: Effect of each rewording on correctness measured by BLEU-1, against the canonical form of its own baseline. A star marks an interval that excludes zero after the correction across the nine rewordings at that evidence level.

Rewording	gold		retrieved, $K = 10$	
	$\hat{\tau}$	95% CI	$\hat{\tau}$	95% CI
A1 lexical	-0.031*	[-0.049, -0.013]	-0.017	[-0.031, -0.003]
A2 syntactic	+0.011	[-0.003, +0.025]	+0.007	[-0.009, +0.022]
A3 format	-0.002	[-0.020, +0.015]	-0.001	[-0.011, +0.009]
B1 lexical	+0.047*	[+0.027, +0.068]	+0.063*	[+0.037, +0.089]
B2 syntactic	+0.034*	[+0.018, +0.050]	+0.035*	[+0.016, +0.054]
B3 format	+0.042*	[+0.026, +0.058]	+0.048*	[+0.028, +0.069]
C1 lexical	+0.010	[+0.002, +0.018]	+0.006	[+0.001, +0.010]
C2 syntactic	+0.029*	[+0.022, +0.037]	+0.022*	[+0.017, +0.027]
C3 format	+0.050*	[+0.041, +0.059]	+0.051*	[+0.044, +0.058]

Of the rewordings of C , the syntactic and format ones raise it, the format one most, by 0.050 to 0.066; no estimate for the lexical rewording $C1$ survives correction. For baseline A , the lexical rewording $A1$ lowers it: by 0.022 on ROUGE-L and 0.031 on BLEU-1 at the gold level, and by 0.018 on ROUGE-L at the retrieved level. The BLEU-1 estimate there, -0.017 , misses the correction. The effects are small compared to those on adherence, a few points against tens of points, but they are consistent.

6.3 The Verdict on the Hypothesis

Eighty-four estimates were made across the five measures of Section 5.7, and thirty-five of them exclude zero after correction: eleven on adherence, twenty-three on the two correctness metrics and one on faithfulness. H_0 states that prompts stating the same requirements in different wordings produce the same distribution of answer quality, so that $\tau_j = 0$ for every rewording, in every baseline, on every measure and at both evidence levels. That is rejected. H_1 claims that at least one rewording changes answer quality, and seven of the nine rewordings do, most of them in both cells.

The rejection does not rest on the single faithfulness result. It rests on instruction adherence, where the effects are large and present in both directions, and on correctness, where they are small but consistent. On both, most effects replicate across the two evidence levels.

That replication also answers the secondary question of Section 5.1, which asked whether the answer to the main question survives a change in the quality of the evidence. It does. Five of the six re-

wordings of adherence move it at both evidence levels, and their sizes are close: B1 gains 0.223 on gold and 0.261 on the retrieved context, C2 loses 0.457 and 0.424. Giving the generator the real output of the retriever, distractors included, rather than only the passages the annotation marks as supporting, does not remove the effect of how the prompt is worded. The correctness effects behave the same way: every rewording with a ROUGE-L effect that survives correction at one level has one at the other.

6.4 Summary of Findings

This thesis asked how far systematic changes in the system prompt influence the behaviour and outcomes of a RAG system when the user input, the knowledge base, the retrieval mechanism and the underlying language model are held fixed. The findings suggest that they influence it considerably.

Twelve system prompts were built from three published baselines. Each baseline was written in its canonical published form and three rewordings that change the vocabulary, the sentence structure or the layout. Every wording states the same requirements. Each prompt was run over 100 MS MARCO queries at two evidence levels with three repetitions. That produced 6,768 answers, and every answer was scored on five measures.

The clearest result is on instruction adherence. Eleven of the twelve intervals exclude zero after correction, with five of the six rewordings moving adherence at both evidence levels. The direction of the effect is not fixed. Every rewording of baseline *B* scored higher than its

published form. The lexical and syntactic rewordings of baseline *C* scored far lower. On faithfulness and answer relevance no effect was found that survives correction, with one exception in one cell that does not replicate in the other. Correctness, measured by ROUGE-L and BLEU-1, moved as well, in smaller steps. Every rewording of baseline *B* and the syntactic and format rewordings of *C* raised it and the lexical rewording of *A* lowered it.

Taken together, these results reject H_0 . Prompts that state the same requirements in different words produced measurably different behaviour from the system tested here. That places the wording of a system prompt among the parameters worth choosing with care.

What the design does not establish. Four limits bound these results. The first is the collection: every figure is conditional on MS MARCO, whose queries are short and mostly factoid, so the effects indicate what may be expected elsewhere without establishing it. The second is the size of effect the design can find. With 88 queries at the retrieved level and 100 at the gold level, a paired comparison at the 5% level detects a standardised effect of about 0.30 and 0.28 respectively, and about 0.39 and 0.37 after correction. Three repetitions per cell instead of five, and a decoding temperature of 1.0 instead of a lower one, both widen the variation within a cell, so a measure that did not move is not thereby shown to be unaffected. The third concerns correctness. The measure fixed before the run was containment of the reference answer, which scored a mean of 0.052 on MS MARCO’s sentence-length references and could not have detected any change. It was replaced by ROUGE-L and BLEU-1 after that result was known, so the replacement was chosen with the data in view. Both metrics reward overlapping words rather than truth and favour answers close

in length to the reference, and the correctness effects follow answer length in every baseline. The fourth is the judge. Faithfulness and answer relevance are scored by a language model, the agreement study against human labels was not carried out, and an audit found that its output cap truncated the longest claim lists and that it is not fully deterministic at temperature 0. The two measures the verdict rests on, correctness and instruction adherence, are computed by program and do not depend on it.

Bibliography

- Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*.
- Cheerla, C. (2025). Advancing retrieval-augmented generation for structured enterprise and internal data.
- Chen, J., Lin, H., Han, X., and Sun, L. (2024). Benchmarking large language models in retrieval-augmented generation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38:17754–17762.
- Cheng, Z. and Mastropaolo, A. (2026). An empirical study on the effects of system prompts in instruction-tuned models for code generation.
- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization.
- Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2023). Ragas: Automated evaluation of retrieval augmented generation.
- Friel, R., Belyi, M., and Sanyal, A. (2024). RAGBench: Explainable benchmark for retrieval-augmented generation systems.
- Gao, L., Ma, X., Lin, J., and Callan, J. (2023a). Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st*

Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers, pages 1762–1777.

Gao, T., Yen, H., Yu, J., and Chen, D. (2023b). Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. (2023c). Retrieval-augmented generation for large language models: A survey.

He, J., Zhang, B., Rouhizadeh, H., Chen, Y., Yang, R., Lu, J., Chen, X., Liu, N., and Teodoro, D. (2025). Retrieval-augmented generation in biomedicine: A survey of technologies, datasets, and clinical applications.

Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020). The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*.

Hua, A., Tang, K., Gu, C., Gu, J., Wong, E., and Qin, Y. (2025). Flaw or artifact? rethinking prompt sensitivity in evaluating LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Jeong, S., Baek, J., Cho, S., Hwang, S. J., and Park, J. C. (2024). Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Volume 1: Long Papers*, pages 7036–7050.

- Jin, B., Zeng, H., Yue, Z., Yoon, J., Arik, S., Wang, D., Zamani, H., and Han, J. (2025). Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning.
- Johnson, J., Douze, M., and Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781. Association for Computational Linguistics.
- Kostić, B., Fallon, C., Risch, J., and Löser, A. (2026). Same meaning, different scores: Lexical and syntactic sensitivity in LLM evaluation. To appear in the Proceedings of the 2026 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC).
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.
- Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain. Association for Computational Linguistics.
- Lin, Y.-H., Chen, Q.-H., Cheng, Y.-J., Zhang, J.-R., Liu, Y.-H., Hsia, L.-Y., and Chen, Y.-N. (2025). LLM inference enhanced by external knowledge: A survey.

- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Lu, S., Schuff, H., and Gurevych, I. (2024). How are prompts different in terms of sensitivity? In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 5833–5856.
- Malkov, Y. A. and Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836.
- Mao, Y., Dong, X., Xu, W., Gao, Y., Wei, B., and Zhang, Y. (2024). FIT-RAG: Black-box RAG with factual information and token reduction.
- Mu, N., Lu, J., Lavery, M., and Wagner, D. A. (2025). A closer look at system prompt robustness.
- Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwary, S., Majumder, R., and Deng, L. (2016). MS MARCO: A human generated MACHine Reading COMprehension dataset.
- Niu, C., Wu, Y., Zhu, J., Xu, S., Shum, K., Zhong, R., Song, J., and Zhang, T. (2024). RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*.

- Pan, J. J., Wang, J., and Li, G. (2024). Survey of vector database management systems. *The VLDB Journal*, 33(5):1591–1615.
- Pandya, H. A. and Bhatt, B. S. (2021). Question answering survey: Directions, challenges, datasets, evaluation matrices.
- Papineni, K., Roukos, S., Ward, T., and Zhu, W.-J. (2002). BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA. Association for Computational Linguistics.
- Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, 2 edition.
- Pipitone, N. and Alami, G. H. (2024). LegalBench-RAG: A benchmark for retrieval-augmented generation in the legal domain.
- Qin, R., Wang, Q., Wang, T., Wei, Z., and Shen, W. (2026). Evaluating and explaining prompt sensitivity of LLMs using interactions.
- Robertson, S. and Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.
- Ruiz, F. J. and Cardona-Betancourt, V. (2026). Prompt carefully! ChatGPT displays rule-based insensitivity to contingencies. *Journal of Contextual Behavioral Science*, 41:101012.
- Saad-Falcon, J., Khattab, O., Potts, C., and Zaharia, M. (2024). ARES: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Lin-*

guistics: Human Language Technologies (NAACL-HLT), pages 338–354.

Salinas, A. and Morstatter, F. (2024). The butterfly effect of altering prompts: How small changes and jailbreaks affect large language model performance. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Salton, G., Wong, A., and Yang, C. S. (1975). A vector space model for automatic indexing. *Communications of the ACM*, 18(11):613–620.

Sciar, M., Choi, Y., Tsvetkov, Y., and Suhr, A. (2024). Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*.

Singh, A., Ehtesham, A., Kumar, S., Khoei, T. T., and Vasilakos, A. V. (2025). Agentic retrieval-augmented generation: A survey on agentic RAG.

Wahle, J. P., Ruas, T., Xu, Y., and Gipp, B. (2024). Paraphrase types elicit prompt engineering capabilities. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 11004–11033, Miami, Florida, USA. Association for Computational Linguistics.

Wang, M., Yao, Y., Xu, Z., Qiao, S., Deng, S., Wang, P., Chen, X., Gu, J.-C., Jiang, Y., Xie, P., Huang, F., Chen, H., and Zhang, N. (2024). Knowledge mechanisms in large language models: A survey and perspective.

Yan, S.-Q., Gu, J.-C., Zhu, Y., and Ling, Z.-H. (2024). Corrective retrieval augmented generation.

- Ying, J., Cao, Y., Xiong, K., Cui, L., He, Y., and Liu, Y. (2024). Intuitive or dependent? investigating LLMs’ behavior style to conflicting prompts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*, pages 4221–4246.
- Yu, H., Gan, A., Zhang, K., Tong, S., Liu, Q., and Liu, Z. (2024). Evaluation of retrieval-augmented generation: A survey.
- Zhan, P., Xu, Z., Tan, Q., Song, J., and Xie, R. (2024). Unveiling the lexical sensitivity of LLMs: Combinatorial optimization for prompt enhancement. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5128–5154, Miami, Florida, USA. Association for Computational Linguistics.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.
- Zheng, M., Pei, J., Logeswaran, L., Lee, M., and Jurgens, D. (2024). When “a helpful assistant” is not really helpful: Personas in system prompts do not improve performances of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 15126–15154.
- Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., and Hou, L. (2023). Instruction-following evaluation for large language models.

Zhuo, J., Zhang, S., Fang, X., Duan, H., Lin, D., and Chen, K. (2024). ProSA: Assessing and understanding the prompt sensitivity of LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2024*.